

Intent-Spezifikation — ReqPOOL Software Estimation Manager

Version 1.0 · Status: bereit zur agentischen Umsetzung · Einheit: PT = Personentage (1 PT = 8 h) · Bezeichner in JSON/Code englisch

1 · Zweck & Kern-Intent

Das Werkzeug wandelt acht objektive Größenparameter eines Softwarevorhabens in eine **parametrisierte, nachvollziehbare Aufwandsschätzung** (PT, inkl. Bandbreite) und eine daraus abgeleitete **Scope-Beschreibung**. Verbindlich sind die Kontrakte (Abschnitte 5, 6) und der **goldene Testfall** (Abschnitt 7); die technische Umsetzung ist frei wählbar.

- **Parametrisiert & kalibrierbar:** alle Treiber, Faktoren und Aufschläge liegen in einer externen Kalibrierung — im Code keine fest verdrahteten Schätzwerte.
- **Deterministisch & nachvollziehbar:** gleiche Eingabe + Kalibrierung → identische Ausgabe; jede Zahl ist auf „Eingabe × Kalibrierwert“ rückführbar.
- **Anschlussfähig:** JSON-Ausgabe zur direkten Weiterverarbeitung durch ein nachgelagertes Angebots-/Honorarwerkzeug.

2 · Eingabeparameter — 5 Size Driver (additiv) + 3 Modifikatoren (multiplikativ)

Parameter	Typ / Bereich	Wirkprinzip	Bedeutung
pages	int ≥ 0	Size Driver	Oberflächen / Seiten (UI-Screens)
useCases	int ≥ 0	Size Driver	fachliche Use Cases / Abläufe
businessObjects	int ≥ 0	Size Driver	Geschäftsobjekte (Entitäten im Datenmodell)
interfaces	int ≥ 0	Size Driver	externe Schnittstellen / Integrationen
batches	int ≥ 0	Size Driver	Batch-/Hintergrundverarbeitungen
languages	int ≥ 1	Modifikator	unterstützte Sprachen → Lokalisierungsfaktor
roles	int ≥ 1	Modifikator	Benutzerrollen → Berechtigungsfaktor
users	int ≥ 0	Modifikator	Nutzeranzahl → Skalierungsfaktor (gestuft)

Erweiterbar: weitere Treiber/Modifikatoren lassen sich allein über die Kalibrierung ergänzen, ohne Änderung der Berechnungslogik.

3 · Berechnungslogik (deterministisch, in dieser Reihenfolge)

E[d]	=	count[d] × rate[d] × complexity[d]	# je Size Driver d
f_language	=	1 + (languages - 1) × language.increment	
f_role	=	1 + (roles - 1) × role.increment	
f_scale	=	Faktor der ersten Nutzer-Stufe mit maxUsers ≥ users	
E_adj[d]	=	E[d] × [f_m d ∈ modifier[m].appliesTo]	# zutreffende Modif.
RealizationCore	=	Σ E_adj[d]	
Realization	=	RealizationCore × f_scale	# scale.appliesTo = "total"
Subtotal	=	Realization × (1 + Σ Phasenaufschläge)	
Total	=	Subtotal × (1 + Risikoreserve)	
low / high	=	Total × (1 + Band.low) / Total × (1 + Band.high)	

4 · Kalibrierung — Default-Startwerte (vor Produktiveinsatz zu kalibrieren)

Size Driver	rate (PT/Einheit)	Skalierungsstufe (users)	f_scale
pages	2,0	≤ 50	1,00
useCases	3,0	≤ 500	1,05
businessObjects	1,5	≤ 5 000	1,10
interfaces	5,0	≤ 50 000	1,20
batches	3,0	> 50 000	1,30

Modifikator	Wert	Phasenaufschlag (% v. Realization)	Wert
language.increment (→ pages, useCases)	0,08	requirements	15 %
role.increment (→ pages, useCases)	0,05	design	10 %
Risikoreserve	15 %	test	20 %
Bandbreite (low / high)	−20 % / +30 %	deployment	8 %
complexity (je Treiber)	1,0	projectManagement	15 %
		documentation	5 %

Hinweis: Die Werte sind plausible Startwerte zur Veranschaulichung und **müssen vor produktivem Einsatz mit ReqPOOL-Erfahrungswerten (historische Projekte) kalibriert** werden.

5 · Ein-/Ausgabe-Kontrakt (JSON)

Eingabe

```
{
  "outputLanguage": "de|en",
  "parameters": {
    "pages": int≥0, "useCases": int≥0,
    "businessObjects": int≥0,
    "interfaces": int≥0, "batches": int≥0,
    "languages": int≥1, "roles": int≥1,
    "users": int≥0
  }
}
```

Ausgabe

```
{
  "estimate": {
    "unit", "driverEfforts{...}",
    "appliedModifiers{language,role,scale}",
    "realizationCore", "realization",
    "phases{...}", "subtotal",
    "riskContingency", "total",
    "totalRounded",
    "range{low,expected,high}"
  },
  "scope": { "summary", "breakdown[]",
    "inScope[]", "assumptions[]",
    "exclusions[]" }
}
```

6 · Scope-Beschreibung (regelbasiert erzeugt)

summary (Template, DE-Default): „Die Anwendung umfasst {pages} Oberflächen in {languages} Sprache(n), realisiert {useCases} Use Cases auf {businessObjects} Geschäftsobjekten, bindet {interfaces} Schnittstelle(n) an, stellt {batches} Batch-Verarbeitung(en) bereit und ist rollenbasiert für {roles} Rolle(n) sowie {users} Nutzer ausgelegt.“

inScope / assumptions / exclusions werden je Parameter erzeugt (leere Treiber entfallen). Standard-Ausschlüsse: Betrieb/Hosting & Infrastruktur, Lizenz-/Drittsoftware, fachliche Datenmigration (falls nicht als Schnittstelle/Batch modelliert), Hardware, Schulung über Doku hinaus, Wartung nach Go-Live. Bei outputLanguage: "en" analoge englische Templates; Zahlen bleiben unverändert.

7 · Goldener Testfall (verbindlicher Abnahmeanker)

Eingabe: pages 20 · useCases 30 · businessObjects 15 · interfaces 4 · batches 5 · languages 3 · roles 6 · users 2000

Faktoren: $f_{\text{language}} = 1,16$ · $f_{\text{role}} = 1,25$ · $f_{\text{scale}} = 1,10$

driverEfforts (nach Modifikatoren)	PT
pages (58,0) · useCases (130,5) · businessObjects (22,5) · interfaces (20,0) · batches (15,0)	—
RealizationCore	246,0
Realization (× f_{scale} 1,10)	270,6
Subtotal (+ Phasen 73 %)	468,14
Total (+ Risiko 15 %) → gerundet	538 PT
Bandbreite (low / expected / high)	430 / 538 / 700 PT

8 · Nicht-funktionale Intents & Definition of Done

- ✓ Eingabe gegen Schema validiert; klare Validierungsfehler statt stiller Defaults.
- ✓ Berechnung nutzt ausschließlich die externe Kalibrierung (keine Magic Numbers).
- ✓ Ausgabe ist deterministisch und enthält den vollständigen Rechenweg; validiert gegen das Ausgabe-Schema.
- ✓ Bandbreite low/expected/high sowie Scope inkl. exclusions werden ausgewiesen.
- ✓ Kern seiteneffektfrei als `estimate(input, config) → output`; CLI/UI optional darum herum.
- ✓ **Goldener Testfall (Abschnitt 7) als automatisierter Test grün**; README mit Aufruf-Beispiel (JSON ein → JSON aus).

Empfohlene Umsetzung (nicht bindend): library-first, TypeScript/Node (Zod, Vitest) oder gleichwertig Python. Reihenfolge: Schemata & Default-Kalibrierung → Kern/Formel → Scope-Generator → CLI/Ausgabe → goldenen Testfall grün stellen → optional UI/Lokalisierung.